

Table of Contents

Agentic Engineering	1
A Textbook for Engineers Building Software with AI Agents	1
Part I - Foundations	2
Chapter 1. The Paradigm Shift	2
Chapter 2. What Agentic Engineering Is (and Is Not)	4
Chapter 3. Roles and Responsibilities	6
Chapter 4. Core Principles	8
Chapter 5. The Mental Model: Capable but Unaccountable	9
Part II - The Lifecycle	12
Chapter 6. The Agentic Engineering Lifecycle	12
Chapter 7. Writing Specifications That Work	14
Chapter 8. Reviewing Generated Code as Evidence	16
Chapter 9. Verification and Testing	18
Chapter 10. Controlling Mutation and Publication	19
Chapter 11. Handling Failure: Route vs. Goal	21
Part III - Practice	23
Chapter 12. Context and Prompt Engineering for Agents	23
Chapter 13. Safety, Security, and Risk	25
Chapter 14. Tooling and Agent Capabilities	26
Chapter 15. Team and Organizational Adoption	28
Chapter 16. Anti-Patterns	30
Chapter 17. The Maturity Model	32
Part IV - Reference	33
Chapter 18. Templates	33
Chapter 19. Worked Case Studies	34
Chapter 20. The Working Checklist	36
Chapter 21. Glossary	37
Chapter 22. Further Reading	38
Appendices	39
Appendix A - Example Specification Documents	39
Appendix B - Failure-Mode Catalog	40
Appendix C - Exercises	41
References	42

Agentic Engineering

A Textbook for Engineers Building Software with AI Agents

Edition: 2.0 (Textbook)

Audience: Software engineers, technical leads, staff engineers, and engineering managers adopting AI-assisted software development.

Scope: This book describes the paradigm in which AI agents plan, write, test, and iterate on software while a human engineer owns intent, architecture, review, and acceptance. It is both a conceptual treatment and a practical handbook: why the paradigm exists, how to work inside it, and how to recognize and avoid its characteristic failure modes.

How to read this book: Part I builds the conceptual foundation. Part II walks the core lifecycle stage by stage. Part III covers the surrounding practice. Part IV is reference material. If you are short on time, start with Chapter 6 (the lifecycle), Chapter 7 (specifications), and Chapter 8 (review), then return to Part I.

Part I - Foundations

Chapter 1. The Paradigm Shift

Learning objectives. After this chapter you should be able to: (1) name the three eras of code production and what distinguishes each; (2) state what changed in the agentic era and what did not; (3) explain why the scarce resource shifts from typing to judgment.

1.1 From authoring to direction

For decades, the dominant model of software production was authoring: an engineer typed implementation into a text editor, line by line, and the tool (compiler, linter, test runner) checked the result. The engineer’s leverage came from typing speed, memory, and fluency in a language and its libraries.

AI-assisted coding changed this in two stages [1][2][6]:

1. **Completion era.** The AI is an autocomplete engine. It suggests the next token, line, or function. The human still writes, still decides, still owns every keystroke. The AI raises typing throughput but does not change who is responsible [1].
2. **Agentic era.** The AI is a goal-directed agent. It receives a specification, plans an approach, writes files, runs tests, reads errors, and iterates until it believes the goal is met. The human’s job moves up a level: from producing code to directing, reviewing, and accepting code [2].

This second stage is agentic engineering, also called spec-driven development or agentic coding [2][3][4]. It is not “vibe coding plus guardrails”; it is a different division of labor with a different failure profile and a different set of skills.

1.2 Three eras of code production

Era	The human does	The machine does	The scarce skill
Authoring	Writes every line	Compiles, links, runs tests	Typing fluency, memory
Completion	Writes; accepts/rejects suggestions	Predicts next tokens	Editing speed
Agentic	Specifies, reviews, accepts	Plans, writes, tests, iterates	Judgment and verification

The table makes the shift legible: in each era, one kind of labor is automated away, and a different kind becomes the bottleneck. The agentic era does not eliminate engineering labor; it relocates it from the keyboard to the review chair.

1.3 What actually changed

The shift is not cosmetic. Each of the following moves from the human to the agent:

- Production of implementation: the agent writes the code.
- First-pass testing: the agent runs checks and reads failures.
- Local iteration: the agent diagnoses and repairs its own errors.
- Summarization: the agent reports what it changed and why.

And each of the following stays, or grows, with the human:

- Intent: what the software must do, and why.
- Architecture: boundaries, data model, interfaces, non-functional constraints.
- Acceptance: the criteria by which a change is judged done.
- Accountability: who answers for what ships.

1.4 Prior analogies, and their limits

The agentic shift resembles earlier jumps in the history of software, and the analogy is instructive but only partial.

- **Compilers.** Moving from machine code to a high-level language moved the engineer from specifying instructions to specifying intent, and the compiler’s output was trusted only after decades of maturity. The lesson: abstraction moves trust, it does not remove it.
- **High-level languages and frameworks.** These raised the level of expression but did not remove the need to understand what the abstraction does underneath when it fails.
- **Automated testing.** Tests changed where confidence lives, from the author’s belief to executable evidence. Agentic engineering generalizes this: confidence lives in evidence, not in the producer’s claim.

Each analogy teaches the same thing: a new layer of automation changes where human judgment is spent and where failures hide; it does not remove the need for judgment or for evidence.

1.5 What stays true

Despite the shift, the fundamentals of engineering do not disappear:

- Correctness still must be demonstrated, not asserted.
- Requirements still must be precise enough to be testable.
- Changes still carry risk proportional to the surface they touch.
- Releasing broken software still costs more than catching it early.
- A human still owns the outcome.

What changes is where the human's effort goes. The scarce resource is no longer typing bandwidth; it is judgment: the ability to specify intent precisely, to read generated code skeptically, and to accept or reject on the basis of evidence [3].

1.6 The one-sentence definition

Agentic engineering is the discipline of specifying desired software outcomes precisely, delegating implementation and iteration to an AI agent, and accepting results only when the source and its execution evidence satisfy explicit acceptance criteria [3].

Key takeaways. The agentic era automates implementation and local iteration, and relocates human effort to specification, review, verification, and acceptance. The scarce skill becomes judgment, not typing.

Chapter 2. What Agentic Engineering Is (and Is Not)

Learning objectives. After this chapter you should be able to: (1) define agentic engineering precisely; (2) place it on the spectrum from vibe coding to spec-driven development; (3) identify three things the paradigm is commonly mistaken for; (4) judge when the paradigm is appropriate.

2.1 Definition

Agentic engineering is the discipline of specifying desired software outcomes precisely, delegating implementation and iteration to an AI agent, and accepting results only when the source and its execution evidence satisfy explicit acceptance criteria [3].

Three words carry the weight: specifying, delegating, and accepting. The discipline lives in the specification you write, the delegation you scope, and the acceptance you gate on evidence.

2.2 The spectrum

Agentic engineering is best understood on a spectrum of how much structure surrounds the delegation:

Practice	Specification	Review	Verification	Appropriate for
Vibe coding	Implicit, conversational	Light or none	Rarely	Prototypes, exploration
Spec-driven development	Written outcome and criteria	Regular	Usually	Features, bug fixes

Practice	Specification	Review	Verification	Appropriate for
Agentic engineering	Precise, verifiable spec; mutation and publication intent; evidence-gated acceptance	Systematic, risk-proportioned	Always, proportional to risk	Production software, correctness-critical work

Vibe coding is not the enemy; it is the entry layer. It raises the floor: anyone can get a working draft. Agentic engineering raises the ceiling: it preserves the quality bar of professional software at production scale [3]. The failure is using one where the other is required.

2.3 What it is not

The paradigm is frequently mistaken for things it is not. Rejecting the caricatures is part of understanding the real thing.

- **It is not no code.** Code still gets written; it is written by an agent. The engineer must still be able to read and reason about that code.
- **It is not no review.** The opposite: review becomes the central act. What is automated is typing, not scrutiny.
- **It is not no accountability.** Accountability does not transfer to the agent, which cannot bear consequences. The human remains accountable for what ships.
- **It is not the agent is always right.** The agent is a statistical system with no skin in the game. Its output is a hypothesis to be tested, not a result to be trusted.

2.4 The failure profile

Agentic failures differ from traditional failures in kind, and knowing this in advance is what the rest of the book prepares you for.

Traditional failure	Agentic failure
A bug from a mistaken mental model of the code	Plausible-looking code that silently violates the specification
A missed edge case the author forgot	An edge case the spec never named, that the agent silently filled in
A slow developer who ships less	A fast agent that ships more, including more defects
A bug caught by the author's intuition	A bug invisible to the agent's confidence, which is uniformly high

The common thread: the agent's output is uniformly confident and variably correct, and its failures are failures of specification and verification rather than of effort. The entire workflow exists to make those failures detectable before they ship.

2.5 When to use it

The paradigm is a fit decision, not a religion.

Context	Verdict	Reasoning
Greenfield prototype	Use heavily	Low downside, high iteration value
Well-specified bug fix	Use	Outcome is falsifiable
Behavior-preserving refactor	Use with care	Requires an explicit preserve-behavior constraint and strong tests
New feature with clear acceptance	Use	Criterion-driven acceptance fits
Correctness-critical algorithm	Use with proof	Require machine-checked proof or exhaustive check
Open-ended product decision	Do not delegate intent	The agent must not decide what the product should be
Ill-specified migration touching shared state	Use with care	Requires deep review and rollback planning

Key takeaways. Agentic engineering is a discipline of specification, delegation, and evidence-gated acceptance. It is not the removal of code, review, or accountability. Its characteristic failures are confident-but-wrong outputs, which is why the rest of the book is about making wrongness detectable.

Chapter 3. Roles and Responsibilities

Learning objectives. After this chapter you should be able to: (1) state exactly which responsibilities remain with the human and which transfer to the agent; (2) explain the capable-but-unaccountable asymmetry; (3) apply a responsibility matrix to any task.

3.1 The division of labor

A clean separation of roles is the foundation of the paradigm. When roles blur, accountability disappears. The division is not “human thinks, agent types”; it is sharper than that: the human owns the questions, the agent owns the attempts, and only the human owns the answers. In the practitioner framing, the human acts as architect, reviewer, and decision-maker while the agent executes, tests, and refines [3].

3.2 The human engineer owns

Area	Responsibility
Intent	What the software must do and why. The human is the source of requirements.

Area	Responsibility
Architecture	The system boundaries, data model, interfaces, and non-functional constraints the agent must respect.
Acceptance criteria	The concrete, checkable evidence a change must produce to be accepted.
Review	Reading the generated source and diff before trusting any claim about it.
Authorization	Whether a change may be made, and whether it may be published.
Accountability	The human is accountable for what ships, regardless of who generated it.

3.3 The AI agent owns

Area	Responsibility
Planning	Decomposing the specification into steps.
Implementation	Writing the code and configuration.
Testing	Running tests, linters, and builds; reading failures.
Iteration	Diagnosing and repairing its own errors.
Reporting	Summarizing what it changed and what evidence it produced.

3.4 The critical asymmetry

The agent is capable but unaccountable. It can produce code, but it cannot bear the consequence of broken code. It can report “tests passed,” but its report is an untrusted claim until the evidence is inspected. It can explain its reasoning, but its explanation is a narrative, not a proof.

This asymmetry is not a defect to be eliminated; it is the defining constraint of the paradigm. The entire workflow exists to manage it.

3.5 A responsibility matrix

Use this matrix, an adaptation of the RACI model, to assign responsibility for any agentic task. R = responsible (does the work), A = accountable (owns the outcome), C = consulted, I = informed.

Activity	Human	Agent	Notes
Define the goal	A, R	C	The agent may help clarify, never decide intent
Write the specification	A, R	C	Acceptance criteria are the human’s deliverable

Activity	Human	Agent	Notes
Plan the approach	C	R, A	Human approves the plan's shape
Write the code	I	R, A	
Run tests and builds	C	R, A	Human interprets failures
Review the diff	A, R	I	
Decide acceptance	A, R	none	Agent's opinion is not a vote
Authorize publication	A, R	none	Per-action, per-turn
Own the shipped result	A	none	Accountability never transfers

Warning. The two most dangerous cells are “Define the goal” and “Decide acceptance.” If the agent is responsible for either, the human has abdicated the role that defines the paradigm. Delegation of implementation is leverage; delegation of intent or acceptance is abdication.

Key takeaways. The human owns intent, architecture, acceptance, review, authorization, and accountability. The agent owns planning, implementation, testing, iteration, and reporting. The asymmetry, capable but unaccountable, is the load-bearing constraint of the whole discipline.

Chapter 4. Core Principles

Learning objectives. After this chapter you should be able to: (1) state the six principles; (2) give a concrete practice and a concrete violation for each; (3) recognize which principle a given failure traces back to.

Six principles govern the practice. They are load-bearing: each maps to a concrete action in the lifecycle, and nearly every failure in this book traces back to the violation of one of them.

Principle 1 - Own intent, architecture, and acceptance

Statement. The human is the authority on what and why; the agent is the executor of how. Never delegate intent; never accept a result without criteria.

In practice. Before delegating, you can state the goal in one falsifiable sentence and the evidence that would prove it met.

Violation. Asking the agent, “What should this feature do?” and then treating its answer as a requirement.

Principle 2 - Specify before you delegate

Statement. Write the specification before the agent writes code. A useful specification states the concrete outcome, the mutation intent, the publication intent, and evidence-based acceptance criteria.

In practice. You can hand a third party the specification and they can check acceptance without asking you questions.

Violation. “Just clean up the codebase” - no bounds, no criteria, no way to tell success from damage.

Principle 3 - Review code as evidence

Statement. Review generated code as evidence, not as ritual. Read the exact source and the post-change diff. Treat the agent’s summary as an untrusted report until the source confirms it.

In practice. You can state, from the source itself, what the change does and what it might break.

Violation. Accepting “tests passed” from the summary without reading the diff or the test results.

Principle 4 - Verify before you accept

Statement. Verification is a completion requirement, never an optional step. Choose checks proportional to the change and the affected surface [3].

In practice. Every acceptance criterion has a corresponding executable check.

Violation. Describing work as done on the agent’s word alone, with no passing evidence.

Principle 5 - Control mutation and publication

Statement. Do not allow mutation of files unless a change was requested. Preserve unrelated work. Never commit, push, or open a pull request without explicit authorization.

In practice. The default for every task is “do not publish,” stated up front.

Violation. The agent pushes to the main branch because you forgot to say not to.

Principle 6 - Treat failure as route failure

Statement. When an approach fails, the route is stalled, not the goal. Pivot to a materially different strategy and record the blocker.

In practice. Every retry differs in kind: a different decomposition, library, technique, or order of work.

Violation. Re-running the same failing attempt with trivial changes, or abandoning the goal because one route failed.

Key takeaways. The six principles - own intent, specify first, review as evidence, verify before accepting, control mutation and publication, treat failure as route failure - are the invariant core. Everything else in this book is elaboration of these six.

Chapter 5. The Mental Model: Capable but Unaccountable

Learning objectives. After this chapter you should be able to: (1) describe the trust model that underlies every other practice; (2) state the default attitude toward any agent claim; (3) run the five-step trust ladder on any piece of agent output.

5.1 The agent as executor, not authority

The single most useful mental model is this: the agent is a capable executor and an untrustworthy authority. It is excellent at doing - planning, writing, running, iterating. It is unreliable at vouching - telling you that what it did is correct, safe, and complete.

5.2 The trust ladder

Given that the agent is an untrustworthy authority, how do you ever accept anything it produces? The answer is a graduated climb, never a leap. Run every piece of agent output up the following five-rung ladder, and refuse to skip a rung:

1. **Default skepticism.** The starting state for any agent claim - “tests pass,” “the fix is complete,” “no behavior changed” - is *unverified*, not *false*. You owe the claim neither belief nor disbelief, only inspection.
2. **Inspect the artifact.** Read the source, the diff, the configuration, the test output. A claim is only as good as the artifact it points at. “It passed” is a pointer to a log file; go read the log file.
3. **Reproduce independently.** Where practical, run the check yourself rather than accepting the agent’s transcript of the check. An agent can misreport, truncate, or misread its own output. Your own execution is independent evidence.
4. **Cross-check against the specification.** Do not ask “does this look right?” Ask “does this satisfy each acceptance criterion, criterion by criterion?” The spec, not the agent’s confidence, is the yardstick.
5. **Accept.** Only after rungs 1-4 have all cleared do you accept, and even then you record *what* you accepted and *on what evidence*. Acceptance is a decision with a paper trail, not a feeling.

The ladder is the operational form of the mental model. Every later chapter - specification, review, verification, mutation control, failure handling - is one rung of this ladder elaborated into a full practice.

5.3 Confidence is not correctness

The agent’s output comes wrapped in a uniform coat of confident prose. This is a property of the system, not a signal about the work. A language model that is wildly wrong about a detail will, in the typical case, express that wrongness with exactly the same tone as a correct answer [5].

Three consequences follow:

- **Calibration is not assumed.** Do not infer “the agent seems unsure, so it is probably wrong” nor “the agent seems sure, so it is probably right.” Confidence text is noise. Look only at evidence.
- **Silent omissions are the default failure.** The dangerous case is not the agent saying “I don’t know” - it is the agent silently filling a gap the specification never addressed, and reporting success. This is why specifications must be complete enough that the agent has nothing to silently invent.
- **Politeness is not agreement.** If you push back, a compliant agent may reverse itself without new evidence, agreeing with your correction whether or not the correction was right.

Treat an agent's agreement as worth nothing by itself; treat your own claim as equally in need of evidence.

5.4 The agent's strengths and blind spots

A realistic model of the agent - what it is genuinely good at and where it reliably fails - lets you spend your attention where it pays.

Strengths (delegate these freely):

- Translating a precise specification into working code.
- Boilerplate, scaffolding, migrations, configuration, and tests.
- Mechanical refactors and renames with clear invariants.
- Running checks and reading their output.
- Iterating on its own errors within a bounded, well-specified loop.
- Producing drafts fast enough that you can afford to discard them.

Blind spots (verify these yourself):

- Knowing what the product *should* do in the face of ambiguity.
- Detecting that it has silently changed unrelated behavior.
- Judging whether a dependency or a pattern is safe to adopt.
- Recognizing when it is out of its depth rather than improvising.
- Bearing the consequence of a defect - it cannot, so it does not.

The asymmetry is stable and structural. It will not be fixed by the next model version; it will only move the boundary. The discipline of the paradigm is to exploit the strengths and defend against the blind spots, in every single task.

5.5 Consequences for the workflow

The mental model is not a slogan; it dictates the shape of the workflow. From it derive, directly:

- Because the agent is an untrustworthy authority, **the human writes the specification** and the acceptance criteria (Chapter 7).
- Because the summary is an untrusted report, **the human reviews the source and the diff** (Chapter 8).
- Because confidence is not correctness, **acceptance is gated on executable evidence** (Chapter 9).
- Because the agent cannot bear consequences, **mutation and publication require explicit human authorization** (Chapter 10).
- Because the agent silently fills gaps, **failure is diagnosed as a specification or route problem** (Chapter 11).

Key takeaways. Hold one model in your head at all times: *capable executor, untrustworthy authority*. Accept nothing on the agent's word; climb the trust ladder from skepticism through inspection, reproduction, and cross-checking to a recorded acceptance. Confidence text is noise; evidence is signal. The rest of this book is the elaboration of this single idea into a repeatable practice.

Part II - The Lifecycle

Chapter 6. The Agentic Engineering Lifecycle

Learning objectives. After this chapter you should be able to: (1) draw the core loop and name its five stages; (2) state the entry and exit criteria for each stage; (3) identify where the human must intervene and where the agent may run autonomously.

6.1 The core loop

Every agentic task, from a one-line fix to a multi-week feature, passes through the same five stages:

1. **Specify** - the human writes what must be true and how it will be judged.
2. **Delegate** - the human hands the specification to the agent, with bounds and authorization.
3. **Review** - the human reads the source, diff, and evidence the agent produced.
4. **Verify** - the human (and the agent, independently) runs checks proportional to risk.
5. **Accept or reject** - the human decides on evidence, then either ships or re-enters the loop.

The loop is a loop, not a pipeline: rejection at stage 5 returns to stage 1 or 2 with a sharper specification or a different route, never with the same unchanged attempt. That return arc is the whole of Chapter 11.

6.2 Stage 1 - Specify

The human writes the specification. It records four things: the concrete **outcome**, the **mutation intent** (what may change and what must not), the **publication intent** (whether commit, push, or pull request is permitted), and **acceptance criteria** - the evidence that proves the outcome was reached.

A specification is complete when a third party could check acceptance without asking you a single clarifying question. If you cannot write that sentence, you are not ready to delegate. Chapter 7 is devoted to this stage.

6.3 Stage 2 - Delegate

The human hands the specification to the agent with explicit bounds: which files or modules are in scope, which tools the agent may use, and what it may *not* do. Delegation is scoped, not open-ended. “Do whatever it takes” is not delegation; it is abdication.

Within the delegated scope, the agent runs autonomously: it plans, writes, tests, reads errors, and iterates. The human does not micro-manage individual keystrokes - that would defeat the purpose - but does hold the boundary and the exit conditions.

6.4 Stage 3 - Review

The human reads the artifact as evidence. This means the diff and the surrounding context, not the agent’s prose summary. The review asks three questions: (1) does the change do what the spec asked? (2) does it do anything *else* - scope creep, behavior change, side effects? (3) are the stated assumptions and edge cases actually handled? Chapter 8 is devoted to this stage.

6.5 Stage 4 - Verify

The human confirms, with executable evidence, that the acceptance criteria hold. The agent's "tests passed" is a claim; the log, the exit code, the proof certificate is the evidence. Verification is proportional to the surface the change touches: a doc typo needs a read-back; a concurrency change needs tests, possibly a proof, and a careful look at the callers. Chapter 9 is devoted to this stage.

6.6 Stage 5 - Accept or reject

On the basis of the evidence, the human makes a recorded decision. **Accept** means the change is done and may proceed to whatever publication was authorized. **Reject** means the loop re-enters, with a concrete, written reason: a specification gap, a route failure, or a verification failure. Rejection is normal and cheap; shipping a defect is expensive.

6.7 Entry and exit criteria

Each stage has a gate. You move to the next stage only when the gate is satisfied.

Stage	Entry criterion	Exit criterion
Specify	A goal exists	Specification is falsifiable and third-party checkable
Delegate	Specification is complete	Agent has run and produced artifacts and claims
Review	Artifacts and claims exist	You can state from the source what changed and what it might break
Verify	Review is complete	Every acceptance criterion has a passing, inspected check
Accept	Verification is complete	A recorded accept/reject decision exists

A stage whose exit criterion is unmet means the loop stalls *at that stage*, and the fix is specific to that stage - you do not paper over a failed review with extra verification, nor a failed specification with a longer agent run.

6.8 The human's intervention points

The human is not a spectator who swoops in at the end. There are exactly three points where the human is irreplaceable, and everything else can be automated away:

- **At specification** - only the human knows what and why.
- **At review** - only the human carries the accountability that makes scrutiny honest.
- **At acceptance** - only the human decides to ship.

Everything between those points - planning, writing, testing, iterating, reporting - is the agent's domain, and the agent should be left to it. The discipline of the paradigm is to be fully present at those three points and fully hands-off between them.

Key takeaways. The lifecycle is Specify - Delegate - Review - Verify - Accept, a loop in which rejection re-enters with a sharper spec or a new route. Every stage has an entry and exit gate, and failure is fixed at the stage it occurs. The human is irreplaceable at exactly three points: specification, review, and acceptance.

Chapter 7. Writing Specifications That Work

Learning objectives. After this chapter you should be able to: (1) name the four components of a complete specification; (2) write acceptance criteria that are executable rather than aspirational; (3) diagnose and repair the six common specification defects; (4) transform a vague request into a delegable specification.

7.1 Why the specification is the whole game

The specification is where quality enters the system. The agent can only be as good as the spec it receives, because every gap in the spec is a gap the agent will silently fill with something plausible-but-wrong. Spend your effort here, before a single line of code is generated, and every downstream stage becomes cheaper. Skimp here, and review and verification become an expensive attempt to reconstruct the intent you failed to write down.

7.2 The four components

A complete specification has four parts, and none is optional:

1. **Outcome.** A concrete statement of what must be true when the work is done, expressed so it can be recognized by an observer who did not write it. “Make the payment flow faster” is not an outcome; “the p95 latency of the checkout POST must be under 400 ms under the load profile in `load/checkout.yaml`” is.
2. **Mutation intent.** What may change, and what must not. List the files or modules in scope, and state the invariants to preserve. “You may modify `src/checkout/*`; you must not change the public API of `PaymentClient`; existing tests in `test/checkout` must continue to pass.”
3. **Publication intent.** Whether the agent may commit, push, or open a pull request, or must stop at a local change for human review. The safe default is *no publication*. This is stated up front, not discovered after the fact.
4. **Acceptance criteria.** The evidence that proves the outcome was reached. Every criterion should be an executable check - a test, a command, a property - not a description of effort.

7.3 Acceptance criteria as executable checks

The single biggest upgrade you can make to a specification is to convert its acceptance criteria from adjectives into checks.

Weak (adjective)	Strong (executable)
“It should be fast”	“p95 latency < 400 ms in the given load test”

Weak (adjective)	Strong (executable)
“It should handle errors”	“On a 5xx from the upstream, the client retries with backoff and surfaces a typed error; covered by <code>TestRetryOn5xx</code> ”
“It should be secure”	“No secret appears in logs or the diff; <code>gitLeaks</code> reports zero findings”
“It should not break things”	“The full existing suite passes; <code>make lint</code> and <code>make build</code> exit 0”

The test is this: if you cannot write down the command whose output would settle the criterion, the criterion is not yet finished. “Handles errors well” cannot be run; “the retry test passes and the error is typed” can.

7.4 The verifiability test

Before delegating, apply three checks to your specification:

- **Falsifiable.** A criterion must be able to fail. “Works correctly” can never fail, so it can never be verified - it is unfalsifiable and therefore not a criterion.
- **Observable.** The evidence must be visible to you, not only to the agent. If the only proof is the agent’s word, the criterion is not yet observable.
- **Decidable.** There must be a procedure that settles pass or fail. If two reasonable engineers could disagree about whether the criterion is met, sharpen it until they cannot.

7.5 The six common specification defects

Nearly every specification failure is one of these six, and each has a repair:

1. **The empty verb** (“fix”, “improve”, “clean up”). Repair: name the concrete outcome and its measure.
2. **The missing boundary** (“do whatever is needed”). Repair: state mutation intent - what is in scope, what must not change.
3. **The unstated invariant** (behavior the author assumed was obvious). Repair: write the preserve-behavior constraints explicitly.
4. **The unfalsifiable criterion** (“robust”, “clean”, “good”). Repair: convert to an executable check.
5. **The unobservable criterion** (proof exists only in the agent’s head). Repair: require evidence you can inspect.
6. **The silent default** (publication or scope assumed). Repair: state publication intent explicitly, defaulting to none.

7.6 Worked example: vague to delegable

Vague request: “Clean up the auth module, it’s a mess.”

Delegable specification:

Outcome. Refactor `src/auth` to reduce duplication and improve test coverage, with no change to external behavior. **Mutation intent.** You may modify files under `src/auth` and `test/auth`.

You must not change the public functions in `src/auth/index.ts` or any other module. Existing tests outside `test/auth` must pass unchanged. **Publication intent.** Stop at a local change; do not commit, push, or open a PR. **Acceptance criteria.** 1. `make lint` && `make build` exit 0. 2. `make test` passes, including the new tests covering previously untested branches in `src/auth/tokens.ts`. 3. The public API of `src/auth/index.ts` is byte-identical before and after (`git diff -- src/auth/index.ts` is empty). 4. Duplicated token-parsing logic is consolidated into a single helper with a test.

Notice what changed: every adjective became a check, every boundary became explicit, and the publication default is safe. This specification can be handed to a third party - or an agent - and acceptance decided without a clarifying question.

Key takeaways. The specification is where quality enters. It has four parts: outcome, mutation intent, publication intent, and acceptance criteria. Acceptance criteria must be falsifiable, observable, and decidable - ideally executable checks. Six defects account for most specification failures, and each has a direct repair.

Chapter 8. Reviewing Generated Code as Evidence

Learning objectives. After this chapter you should be able to: (1) explain why the agent's summary is not evidence; (2) read a diff and its surrounding context systematically; (3) distinguish observed facts from the agent's inferences; (4) run a review checklist that catches scope creep and behavior change.

8.1 The summary is not evidence

When an agent finishes, it hands you a narrative: what it changed, why, and how confident it is. That narrative is a *pointer*, not a *result*. It points at the real evidence - the source, the diff, the test output - which you must go read yourself. Three reasons the summary is insufficient:

- It is generated by the same system that generated the code, so it inherits the same blind spots.
- It describes intent (“I fixed the race”) rather than mechanics (“I replaced the lock with this specific pattern”), and only the mechanics can be checked.
- It is uniformly confident, and confidence is noise (Section 5.3).

The review's purpose is to replace the summary with your own reading of the evidence.

8.2 Read the diff, then the context

Review order matters. Read the **diff first**: it is the smallest complete description of what actually changed, and it contains no prose to be misled by. For each hunk, ask *what* changed and *why* it could matter.

Then read the **context** - the surrounding code, the callers, the tests, the configuration, the error handling. A change that looks innocent in isolation may be dangerous in context: a signature change rippling into a caller; a new dependency pulling in a supply-chain risk; a “minor” tweak to error handling that silently swallows a failure upstream.

8.3 What to look for

Work through a short list of failure classes, in order of danger:

- **Scope creep.** Changes outside the requested surface. Any touched file not named in the mutation intent is a red flag requiring justification.
- **Behavior change under cover.** A “refactor” that also alters a return value, a default, or an edge case. This is the classic way bugs slip through.
- **Silent gap-filling.** The agent invented a requirement the spec never stated, and implemented it without flagging the assumption.
- **Error-handling regressions.** Swallowed exceptions, removed retries, changed timeouts, weakened validation.
- **Security-relevant changes.** New dependencies, changed permissions, added network calls, secrets in code or logs, weakened input validation.
- **Test weakening.** Tests deleted, skipped, or loosened rather than fixed - the agent “making tests pass” by making them easier to pass.

8.4 Facts versus inferences

As you review, keep two columns: **facts** (what the source directly shows) and **inferences** (what you conclude, with a risk of error). “The function now returns `Option`” is a fact. “This will break the caller” is an inference, to be confirmed by reading the caller. “The agent said this is safe” is neither a fact nor your inference - it is an unverified claim.

This discipline matters because review failures compound: a false inference, confidently held, becomes a shipped defect. Write down the inferences that matter and confirm or refute each one against the source before accepting.

8.5 Cross-boundary review

Most real bugs live at boundaries, not inside the changed function. For every change, walk the boundary it touches:

- **Callers.** Who invokes this code, and do their assumptions still hold?
- **Tests.** Do the tests actually pin the behavior the spec cares about, or only the happy path?
- **Configuration.** Did defaults, environment variables, or feature flags change?
- **Error handling.** What happens on the failure paths - are they still exercised?
- **Public contracts.** Did an interface, schema, or wire format change, and who is on the other side?

8.6 The review checklist

Run this list before any acceptance decision. It is also reproduced in Chapter 20.

1. I read the full diff, not the summary.
2. Every changed file is within the authorized mutation scope.
3. I can state, from the source, what the change does and what it might break.
4. Behavior changes, if any, are intentional and specified.
5. Error paths and edge cases are handled, not just the happy path.
6. No security-relevant change (dependency, permission, secret, validation) escaped notice.
7. Tests were strengthened or correctly updated, never weakened to pass.

8. My material inferences were confirmed against the source, not assumed.

Key takeaways. The agent’s summary is a pointer to evidence, not evidence. Read the diff first, then the context; hunt scope creep, covered behavior change, silent gap-filling, and test weakening. Keep facts and inferences separate, confirm inferences against source, and pay special attention to boundaries.

Chapter 9. Verification and Testing

Learning objectives. After this chapter you should be able to: (1) state why verification is a completion requirement; (2) choose checks proportional to risk; (3) distinguish standard, mathematical, and formal verification; (4) interpret a failing check as evidence rather than an obstacle.

9.1 Verification is a completion requirement

A change is not “done” because the agent says so, nor because it compiles. It is done when the acceptance criteria are met by inspected, executable evidence. Verification is therefore not a step you may skip when in a hurry; it is the gate that separates *produced* from *accepted*. An unverified change is an unaccepted change.

9.2 Proportionality

Verification effort scales with the surface and consequence of the change, not with the difficulty of writing the check.

Change	Minimum verification
Documentation edit	Read-back against the source it documents
Small, isolated bug fix	Targeted test + existing suite
Feature with a new path	Tests for the new path, happy and failure cases, plus the suite
Refactor	Full suite + explicit no-behavior-change check
Concurrency or correctness-critical change	Tests + reasoning, possibly a proof, plus caller analysis
Migration touching shared state	Tests, rollback plan, and staged rollout

The rule of thumb: the check must be strong enough that a plausible-but-wrong version of the change would *fail* it. A check that a wrong answer would also pass is not verification; it is ceremony.

9.3 Levels of verification

Three levels are worth distinguishing, and you reach for each by the risk of the change:

- **Standard.** Native tests, type checks, linting, and builds. This is the everyday floor for code changes.
- **Mathematical.** When a change makes a correctness claim that goes beyond what tests can cover - “this algorithm always terminates,” “this function never returns a negative amount” - you bind the claim to a precise statement and prove it, rather than sampling it with tests.

- **Formal.** When the claim must be beyond dispute - a proof of a safety property, an invariant, a data-structure guarantee - you machine-check the proof in a system such as Lean 4. A proof that does not compile is not a proof; a claim that is only argued in prose is not formally verified.

The discipline: match the evidence to the claim. A claim of “always” is never established by a handful of test cases; it requires proof. A claim of “works for these inputs” is established by tests and nothing more.

9.4 Test selection

Ask three questions to choose tests:

- **What must not regress?** The existing suite is the contract with the past; run it.
- **What is new?** Every new branch, path, and error condition deserves a test that would fail if it were wrong.
- **What boundary was touched?** Add tests at the boundary - callers, error paths, public contracts - not just inside the changed function.

Coverage percentage is a weak proxy. A single test on the one dangerous branch is worth more than a hundred on trivially-true lines. Spend tests where wrongness is expensive.

9.5 Interpreting failures

A failing check is evidence, not an obstacle to be cleared. When a check fails:

1. **Read the failure.** The actual assertion, the actual diff, the actual exit code.
2. **Diagnose before changing.** Is it a defect in the change, a defect in the test, or an unstated assumption now violated?
3. **Fix the cause, not the symptom.** Do not loosen or delete the failing test to make it pass; if the test is wrong, change it deliberately and say why.
4. **Do not repeat an unchanged command.** Re-running the identical failing check produces no new information; it is a stall, not progress.

9.6 The regression trap

The agent, like any engineer under pressure, can be tempted to make failing tests pass by weakening them - deleting an assertion, marking a test skipped, loosening a bound. Treat every change to a test file as a change to the specification, and review it with the same scrutiny as production code. A test that was made easier to pass is not verification; it is the erasure of verification.

Key takeaways. Verification is a completion gate, not an optional step. Scale checks to risk: standard tests for routine changes, mathematical proof for “always” claims, formal proof for claims that must be beyond dispute. Read failures as evidence, diagnose before changing, never weaken a test to pass, and never re-run an unchanged failing command.

Chapter 10. Controlling Mutation and Publication

Learning objectives. After this chapter you should be able to: (1) state the default policy for mutation and publication; (2) scope mutation so unrelated work is preserved; (3) enforce

per-action authorization for commits, pushes, and pull requests; (4) inspect the post-change diff as a gate, not an afterthought.

10.1 The mutation boundary

The agent can write files, and that is exactly the power that must be constrained. The rule is simple: **the agent may mutate files only when a change was requested, and only within the requested scope.** No change request means no mutation; a change request to `src/auth` does not license edits to `src/billing`.

This is not paranoia; it is the difference between a tool and a liability. An agent that may edit anywhere, whenever it “sees fit,” is a source of surprise, and surprise is the enemy of shipped software.

10.2 The authorization model

Two kinds of action need explicit authorization, stated up front in every task:

- **Mutation.** Which files may be created or modified. Default: only those named in the mutation intent.
- **Publication.** Whether the agent may commit, push, or open a pull request. Default: *no publication* - the agent stops at a local change for human review.

Authorization is **per-action and per-turn**, not granted once and forgotten. A task that authorized “commit to a feature branch” does not thereby authorize “push to main” or “open a PR” in a later turn. When the task changes, the authorization is re-established.

10.3 Preserving unrelated work

Two rules protect work you and others already did:

- **Smallest coherent change.** The agent should make the minimal change that satisfies the specification, not “improve things while I’m in there.” Related improvements are a separate task with a separate spec.
- **No collateral edits.** Formatting churn, reformatting, reordering, or “cleanup” in files unrelated to the task is a defect, not a favor. It pollutes the diff and buries the actual change.

When in doubt, the agent should ask rather than expand scope. A clarifying question is cheap; a wide, unwanted diff is expensive to unwind.

10.4 Inspecting the final diff

After any mutation, inspect the final diff before doing anything else. The diff is the record of truth about what changed. Verify:

- Every change maps to a part of the specification.
- Nothing unrelated was touched.
- No test was weakened (Section 9.6).
- No secret, credential, or temporary file was committed.

A diff that you have not read is a change you have not reviewed. Do not commit, push, or otherwise act on a change whose diff you have not personally inspected.

10.5 Publication as a consequential action

Publication is where a local mistake becomes a shared problem. Treat it accordingly:

- **Commit** makes the change part of local history - reversible, but now visible to your future self.
- **Push** shares it with the team - harder to unwind cleanly.
- **Pull request / merge** invites it into the shared mainline - the point of no cheap return.

Each step upward raises the stakes and therefore the bar of authorization. The default posture: the agent stops at a local change, you read the diff, you run verification, and *you* - the accountable human - perform or explicitly authorize each publication step.

10.6 Git discipline in the agentic loop

- Keep agent work on a branch or a clean worktree, never directly on a shared mainline.
- Ask the agent to stage changes narrowly so the diff is reviewable in pieces.
- Never let the agent force-push, rewrite shared history, or resolve conflicts by guessing at intent.
- If the workspace is not a Git repository, capture the change set explicitly (e.g., a manifest of created/modified files) so nothing is lost.

Key takeaways. Mutation requires a change request and a scope; publication defaults to none and is authorized per-action. Preserve unrelated work with the smallest coherent change and no collateral edits. Inspect the final diff before any further step, and treat commit, push, and merge as escalating acts of authorization, not afterthoughts.

Chapter 11. Handling Failure: Route vs. Goal

Learning objectives. After this chapter you should be able to: (1) distinguish a failed route from a failed goal; (2) diagnose the cause before pivoting; (3) design a materially different retry; (4) record blockers and decide when to escalate or stop.

11.1 The distinction

When an agent fails - a proof does not go through, a test keeps failing, a build breaks - the first decision is which of two things failed:

- **The route.** The particular approach, decomposition, library, or sequence of steps was wrong. The goal is still achievable by a different route.
- **The goal.** The specification itself is impossible, contradictory, or ill-posed.

The overwhelming majority of failures are route failures. The discipline is to treat failure as a route failure by default, keep the goal intact, and change the approach - while staying alert for the rare case where the goal itself is the problem.

11.2 Diagnose before pivoting

Do not pivot blindly. Classify the failure first, because each class points to a different fix:

Failure class	Example	Correct response
Specification gap	The agent filled an unnamed edge case	Sharpen the spec (Chapter 7)
Verification failure	A check fails	Diagnose the cause (Section 9.5)
Route failure	The chosen technique does not work	Change the technique
Capability limit	The agent lacks the tool or model ability	Decompose or delegate differently
Goal failure	The spec is contradictory	Reframe the goal with the human

A misdiagnosed failure wastes the retry. Spend a minute classifying before you spend an hour re-running.

11.3 Materially different retries

A retry that is equivalent to the failed attempt is not a retry; it is repetition. Each new attempt must differ **in kind** - at least one of:

- A different decomposition of the problem.
- A different algorithm, library, or tool.
- A different proof technique (induction vs. contradiction vs. construction).
- A different order of work.
- A different boundary: more or fewer subproblems, solved by different means.

The test: if a reviewer cannot tell the difference between attempt N and attempt N+1 from a one-line description, you have not actually retried - you have repeated. Stop, and change something fundamental.

11.4 Decompose and delegate

Many route failures are failures of granularity. A problem that defeats the agent as a monolith may yield when decomposed: split it into independent subproblems, solve the leaves first, and compose upward. This is the general remedy for “the agent flounders on the big thing”:

1. Break the goal into sub-goals with clear dependencies.
2. Solve the independent leaves first.
3. Compose the leaves into the result.
4. Where a sub-goal still resists, decompose it again - and if it remains stuck, that is the signal to escalate rather than loop.

Decomposition is also where parallel delegation pays: independent subproblems can be attacked concurrently, each with its own focused context.

11.5 Escalate, don’t loop

Set a stopping rule before you start: a bounded number of materially-different attempts, after which you stop and escalate rather than continue looping. Escalation means one of:

- A human takes over the stuck piece directly.
- The goal is renegotiated (a weaker but sufficient objective).
- The task is declared blocked with a written reason.

Looping on equivalent attempts is the signature failure of naive agent use. The bounded loop - with a real change of strategy each iteration and a real exit - is the signature of the disciplined practitioner.

11.6 Recording blockers

When a route stalls, record it precisely: what was attempted, what failed, what the error actually said, and what was tried next. A blocker record is an asset, not an admission:

- It prevents repeating the same dead end.
- It is the raw material for the next diagnosis.
- It is the honest status report - “blocked on X” with evidence beats “still working on it” without.

The record needs only four fields: **goal**, **route attempted**, **observed failure**, **next route planned**. That is enough to resume without re-deriving the past.

Key takeaways. Failure is a route failure by default, not a goal failure. Classify the failure before pivoting, then retry with a materially different approach, decompose to a finer granularity when a monolith resists, and stop on a bounded loop to escalate rather than repeat. Record blockers with goal, route, failure, and next route.

Part III - Practice

Chapter 12. Context and Prompt Engineering for Agents

Learning objectives. After this chapter you should be able to: (1) assemble the context an agent needs without drowning it; (2) structure a task for reliable execution; (3) encode constraints and guardrails in the prompt; (4) iterate on prompts by observing where the agent stumbles.

12.1 What the agent needs to know

An agent, unlike a human teammate, has no ambient context. It knows nothing you do not hand it. Before delegating, assemble the minimum context it needs to succeed:

- The **goal** and the **specification** (Chapter 7).
- The **relevant source** - files, interfaces, and callers in scope.
- The **constraints** - invariants, non-functional requirements, things not to do.
- The **authorization** - mutation and publication intent.
- The **definition of done** - the acceptance criteria.

Too little context forces the agent to guess (and it will guess wrong). Too much context dilutes its attention and its ability to follow the core instruction. Hand it what it needs and nothing more.

12.2 Context budgeting

A focused agent outperforms an overloaded one. Budget context deliberately:

- **Trim to the task.** Include the files the task touches, not the whole repository.
- **Prefer excerpts.** Point at specific functions and their callers, not entire modules.
- **State constraints in one place.** Do not bury invariants in paragraphs of prose; list them.
- **Use the spec as the spine.** The specification is the context that matters most; everything else is supporting material.

12.3 Structuring the task

A well-structured task has a recognizable shape:

1. **Objective** - one falsifiable sentence.
2. **Scope and constraints** - what may change, what must not, what to preserve.
3. **Authorization** - mutation and publication limits.
4. **Acceptance criteria** - the executable checks that define done.
5. **Output contract** - what the agent must report back: the change, the evidence, the limitations.

When the agent returns, you should be able to check the result against items 1-4 and verify the report against item 5. If the output contract is unstated, the agent decides what to report, and it will report the favorable parts.

12.4 Constraints and guardrails in the prompt

Guardrails are constraints you write down because the agent cannot infer them:

- **Do-nots** are as important as dos. “Do not change the public API,” “do not commit,” “do not weaken tests.”
- **Preserve-behavior** for any refactor: “external behavior must be byte-for-byte unchanged.”
- **Stop conditions** for open-ended work: “if you cannot meet criterion 3, stop and report the blocker rather than improvising.”
- **Evidence over assertion** for reporting: “report the exact command and its exit code, not a paraphrase.”

A guardrail unstated is a guardrail absent. The agent will not volunteer to constrain itself.

12.5 Examples and few-shot guidance

For repetitive or tricky tasks, a worked example of the desired output is worth more than prose instruction. Show the agent one good specification, one good review note, or one good report, and it will imitate the shape. Few-shot works best when the examples are:

- **Real**, not fabricated to look tidy.
- **Representative** of the hard cases, not the easy ones.
- **Consistent** in format, so the shape is what transfers.

12.6 Iterating on prompts

Prompts are not written once; they are debugged. Treat them like code:

- **Observe where the agent stumbles** and encode the correction into the prompt. If it keeps changing the public API, add an explicit do-not; if it keeps under-reporting, tighten the output contract.
- **Keep a prompt changelog** for tasks you repeat. Each revision records what failure it fixes.
- **Do not over-fit.** A prompt tuned to one instance becomes brittle; prefer constraints that express the invariant, not the specific incident.

Key takeaways. The agent has no ambient context; hand it exactly what it needs. Budget context to the task, structure the task with objective, scope, authorization, criteria, and output contract. State do-nots and guardrails explicitly, use real examples for repetitive work, and iterate on prompts by fixing observed failures.

Chapter 13. Safety, Security, and Risk

Learning objectives. After this chapter you should be able to: (1) name the security-relevant changes to watch for in a diff; (2) protect secrets and credentials from entering code or history; (3) recognize prompt-injection and data-handling risk; (4) scale review to the risk of the change.

13.1 The security review is part of code review

An agent that writes code can, without malice, write insecure code: it can add a dependency with a known vulnerability, weaken input validation, log a secret, or introduce an injection point. Security review is not a separate activity bolted onto the end; it is a lens applied during the normal review (Chapter 8), weighted by risk.

13.2 The security-relevant change list

When reading a diff, pause on any of these:

- **New dependencies.** Every added library is added attack surface and added supply-chain risk. Verify provenance, popularity, maintenance, and known advisories before accepting.
- **Changed permissions or auth.** Any edit to authentication, authorization, or access control deserves line-by-line scrutiny.
- **Secrets.** Credentials, tokens, keys, or secrets in code, logs, or the diff. Automate detection (`gitleaks`, `trufflehog`) and treat a finding as a blocker.
- **Network and data flow.** New endpoints, new outbound calls, new data passed across trust boundaries.
- **Input validation.** Weakened or removed validation, deserialization of untrusted data, SQL built by string concatenation, command execution with user input.
- **Error handling.** Swallowed exceptions that hide failures, or verbose errors that leak internals.

13.3 Secrets and credentials

The most common and most damaging agent mistake is leaking a secret. Enforce these rules:

- **Never commit secrets.** Use environment variables or a secret manager; keep `.env` and key files out of the diff (`.gitignore`).

- **Scan before you accept.** Run a secret scanner on the diff as part of verification; a clean scan is an acceptance criterion.
- **Treat a leaked secret as compromised.** Rotate it immediately; do not assume “it was only in a local branch.” History remembers.

13.4 Prompt injection and adversarial input

The agent may read text that is not trustworthy - a web page, a document, a user message, a code comment - and that text may contain instructions aimed at the agent rather than at the task. This is prompt injection. The mitigations mirror classic security hygiene:

- **Treat all external input as untrusted**, including text the agent fetches or files it reads.
- **Constrain what the agent may do with untrusted content:** read and summarize, but never execute instructions found in data.
- **Keep high-consequence actions** (publishing, deleting, sending) behind human authorization so that no injected instruction can trigger them unilaterally.

13.5 Data handling and privacy

If the task involves personal, customer, or confidential data:

- **Do not send sensitive data** to a model or third-party tool unless that is explicitly authorized by policy.
- **Minimize and mask** - hand the agent only what it needs, and redact the rest.
- **Do not let test fixtures embed real data.** Anonymize before use.
- **Treat the agent’s conversation as a data flow**, and govern it like any other.

13.6 Risk-proportioned review

Not every change merits the same scrutiny. A documentation typo is low risk; a payment path or an auth change is high risk. Scale the depth of review and the strength of verification to the consequence of being wrong - this is the same proportionality principle as Chapter 9, applied to security. The highest-risk changes get line-by-line review, strong tests, possibly a proof, and a human second pair of eyes.

Key takeaways. Security review is a lens on ordinary review. Watch for new dependencies, permission changes, secrets, network and data flow, weakened validation, and error-handling regressions. Never commit secrets; scan the diff. Treat all external input as untrusted against prompt injection, keep high-consequence actions behind human authorization, and govern data flows. Scale scrutiny to risk.

Chapter 14. Tooling and Agent Capabilities

Learning objectives. After this chapter you should be able to: (1) enumerate the categories of tools an agent may have; (2) map each tool to the risks it carries; (3) choose a toolset appropriate to the task; (4) recognize capability boundaries and work within them.

14.1 The agent's toolset

An agent is only as capable as its tools, and each tool is a power that carries a matching risk. Tool categories fall into four groups:

- **Reading tools** - search, file read, symbol lookup, reference navigation. Low risk, high value; the backbone of review.
- **Mutation tools** - file write, edit, patch. Medium risk; must be scoped by mutation intent.
- **Execution tools** - running tests, builds, linters, scripts, and compilers. Medium risk; output must be read, not taken on faith.
- **Publication tools** - commit, push, pull request. High risk; require explicit human authorization.

14.2 Reading tools

Reading tools are how the agent learns the codebase before acting. Insist that the agent *uses* them rather than assuming: “inspect the exact source before making precise claims” is a standing instruction. A capable agent reads the file it is about to change, the callers of that file, and the tests that pin its behavior - before it writes a line.

Reading tools carry little risk on their own, but their *absence* is a risk: an agent that edits without reading is an agent guessing.

14.3 Mutation tools

Mutation tools write to the workspace. Govern them with the mutation boundary (Chapter 10):

- Grant edit capability only for the task's scope.
- Require a change request before any write.
- Prefer narrow, reviewable edits over wholesale rewrites.
- Inspect the diff after mutation, every time.

A tool that can edit anywhere is not “more capable”; it is less safe. Capability and safety are in tension, and the disciplined practitioner resolves the tension by scoping.

14.4 Execution tools

Execution tools run the checks that verification depends on. Use them to *reproduce* the agent's claims: run the tests yourself, run the build yourself, inspect the exit code yourself. The agent's transcript of a test run is a claim; your own execution is evidence (Section 5.2).

Execution tools also carry risk when they can affect the outside world - network access, cloud resources, destructive commands. Keep those behind authorization and prefer isolated, disposable execution for anything uncertain.

14.5 Capability boundaries

Every agent has limits, and the limits move with the model and the tools. Know the current boundary and work within it rather than fighting it:

- **Context length** - bounded memory; budget context (Section 12.2).
- **Determinism** - output varies; do not rely on a fragile one-shot.

- **Formal correctness** - an agent can write a proof sketch, but only a proof checker establishes correctness; do not treat prose as proof.
- **Ground truth** - the agent's knowledge is not current and not verified; for facts that matter, require a source, not a recollection.

14.6 Choosing and configuring tools

Choose tools by the task, not by fashion:

- A review task needs reading tools and nothing else.
- An implementation task needs reading + mutation + execution, with publication withheld.
- A correctness-critical task adds a proof tool and a proof checker, and still withholds publication.

Configure tools so that the *default* is safe and the *high-consequence* actions are gated. Safety should not depend on the agent remembering to be careful; it should be a property of the configuration.

Key takeaways. An agent's tools are its capabilities and its risks. Reading tools are the low-risk backbone; mutation tools must be scoped; execution tools must be used to reproduce claims independently; publication tools require human authorization. Know the capability boundaries, and configure tools so safety is the default, not a request.

Chapter 15. Team and Organizational Adoption

Learning objectives. After this chapter you should be able to: (1) sequence an individual's adoption of agentic engineering; (2) install team-level guardrails that make safety the default; (3) choose metrics that measure quality, not volume; (4) navigate the culture change without false promises.

15.1 The individual path

Adoption is gradual, and the order matters:

1. **Start with low-risk tasks** - docs, tests, small isolated bug fixes - where a mistake is cheap.
2. **Build the spec habit** before relying on the agent for anything consequential (Chapter 7).
3. **Build the review habit** - read every diff, every time, until it is automatic (Chapter 8).
4. **Build the verification habit** - require evidence, never the summary (Chapter 9).
5. **Only then** raise the stakes to features, refactors, and correctness-critical work.

The trap is inverting the order: reaching for the hardest, highest-stakes task first and being surprised by the blast radius.

15.2 Pairing and mentoring

Agentic engineering is a skill best learned by watching a practitioner and being watched in turn. In pairing:

- **The senior writes the spec and reviews** while the learner observes how gaps are found.
- **The learner writes the spec and reviews** while the senior checks the gaps the learner missed.

- **Retrospectives** after each task ask: what did the spec miss? what did review catch? what would have shipped without it?

The learning objective is not “make the agent produce code”; it is “make the human reliably catch what the agent gets wrong.”

15.3 Team-level guardrails

Safety must be a property of the system, not of individual vigilance. Install guardrails at the team level:

- **Templates.** A shared specification template (Chapter 18) makes good specs the path of least resistance.
- **Linters and CI.** Static checks run on every change whether or not a human remembers.
- **Secret scanning.** Automated, blocking, on every diff.
- **Review policy.** No merge without a human review of the diff.
- **Publication policy.** Agents default to no publication; CI and branch protection enforce it.

A guardrail in a policy document is a wish; a guardrail in CI is a fact. Prefer the fact.

15.4 Metrics that matter

Measure what the paradigm is for - quality and cycle time - not what is easy to count. Prefer:

- **Escaped defects** (what shipped broken) - the metric that matters most, and the one that tells you whether review and verification are working.
- **Review depth** (were material issues caught pre-merge) over raw review count.
- **Cycle time for a change** (spec-to-accept), which should fall.
- **Specification quality** (did acceptance criteria hold up, or did review repeatedly reconstruct intent).

Distrust these:

- **Lines of code generated** - the agent can generate volume trivially; volume is not progress.
- **Number of prompts or agent turns** - activity, not outcome.
- **Self-reported success rate** - the agent’s confidence is noise (Section 5.3).

15.5 Culture and change management

The hardest part of adoption is not technical; it is the narrative people carry:

- **“It will replace engineers.”** The honest framing: it relocates engineering effort from typing to judgment (Chapter 1). The scarce skill is now verification.
- **“It means no review.”** The opposite: review becomes the central act (Chapter 8).
- **“Vibe coding is enough.”** For prototypes, yes; for production, no (Chapter 2). The discipline raises the ceiling without dismissing the floor.

Adoption succeeds when the organization treats agentic engineering as a *discipline with standards*, not a *shortcut with no rules*.

Key takeaways. Adopt gradually from low-risk to high-risk tasks, building the spec, review, and verification habits in order. Make safety systemic with templates, CI, secret scanning, and

publication policy. Measure escaped defects and cycle time, not generated volume. Frame the change honestly: judgment, not typing, becomes the scarce skill.

Chapter 16. Anti-Patterns

Learning objectives. After this chapter you should be able to: (1) name the seven canonical anti-patterns; (2) recognize each in your own practice; (3) state the corrective for each.

16.1 Trusting the summary

The pattern. Accepting “tests passed” or “all done” from the agent’s prose without reading the diff or the evidence.

Why it fails. The summary inherits the agent’s blind spots and its uniform confidence (Sections 5.3, 8.1).

Correction. The summary is a pointer. Read the diff and reproduce the checks (Chapters 8, 9).

16.2 Delegating intent

The pattern. Asking the agent “what should this feature do?” and treating its answer as the requirement.

Why it fails. The agent cannot bear consequences and has no authority to decide what the product should be (Chapter 3).

Correction. The human writes the outcome and the acceptance criteria; the agent may help clarify, never decide (Chapter 7).

16.3 Skipping verification

The pattern. Declaring work done because it compiles, or because time ran out, without inspected evidence.

Why it fails. Verification is the gate between produced and accepted (Chapter 9). Unverified is unaccepted.

Correction. Every acceptance criterion has an executable, inspected check, run before acceptance.

16.4 Scope creep and over-mutation

The pattern. The agent “improves things while it’s in there” - reformatting, refactoring, and editing files outside the requested scope.

Why it fails. Collateral edits bury the real change and break unrelated work (Chapter 10).

Correction. Mutation intent names the scope; the smallest coherent change is the default; anything wider is a separate task.

16.5 The “one more prompt” loop

The pattern. Responding to every failure by re-prompting with a nudge - “try again,” “be more careful” - rather than changing the approach.

Why it fails. Equivalent retries are repetition, not iteration (Chapter 11). The agent cannot fix a route failure by trying harder on the same route.

Correction. Classify the failure, then change something fundamental - decomposition, technique, or boundary.

16.6 Weakening tests to pass

The pattern. The agent deletes or loosens a failing assertion so the suite turns green.

Why it fails. A weakened test is the erasure of verification (Section 9.6). It converts a caught defect into a shipped one.

Correction. Review test changes like production changes; treat a loosened test as a defect, not a fix.

16.7 Unbounded autonomy

The pattern. Granting the agent mutation and publication power “so it can move fast,” with no per-action authorization.

Why it fails. An unconstrained agent is a source of surprise; publication is the point of no cheap return (Chapter 10).

Correction. Default to no publication; authorize mutation and publication per-action; inspect every diff.

16.8 The catalog at a glance

Anti-pattern	Root principle violated	Correction
Trusting the summary	Review as evidence	Read the diff, reproduce checks
Delegating intent	Own intent and acceptance	Human writes the spec
Skipping verification	Verify before accepting	Executable, inspected checks
Scope creep	Control mutation	Smallest coherent change
“One more prompt” loop	Route vs. goal	Materially different retry
Weakening tests	Verify before accepting	Review tests like code
Unbounded autonomy	Control publication	Per-action authorization

Key takeaways. The seven canonical anti-patterns - trusting the summary, delegating intent, skipping verification, scope creep, the one-more-prompt loop, weakening tests, and unbounded autonomy - each traces to the violation of one core principle, and each has a direct correction. Recognizing the anti-pattern is half the remedy.

Chapter 17. The Maturity Model

Learning objectives. After this chapter you should be able to: (1) place yourself or your team on the five-level maturity scale; (2) name what changes at each level; (3) state the concrete next step to advance; (4) recognize regression signals.

17.1 The five levels

Maturity in agentic engineering is measured by how reliably *defects are caught before they ship*, not by how much the agent produces.

Level	Name	Signature behavior
0	Ad hoc	The agent is prompted informally; results are trusted on the summary
1	Spec-aware	Specifications are written, but acceptance criteria are vague
2	Evidence-gated	Acceptance is gated on inspected checks; diffs are reviewed
3	Systemic	Templates, CI, secret scanning, and publication policy enforce the practice
4	Verified	Correctness-critical claims are proven, not sampled; formal proof where required

17.2 What changes at each level

- **0 to 1:** The team stops delegating intent and starts writing specifications (Chapter 7).
- **1 to 2:** Acceptance criteria become executable, and review becomes a required step (Chapters 8, 9).
- **2 to 3:** Guardrails move from individual discipline into CI, templates, and policy (Chapter 15).
- **3 to 4:** The team matches evidence to claim - proof for “always” claims, tests for “these cases” claims (Chapter 9).

17.3 Advancing

Advancement is deliberate, one level at a time:

1. Identify the current level honestly - ask “what would have shipped broken last month?”
2. Target the single next-level behavior and make it a *default*, not a resolution.
3. Install the guardrail that enforces the new default (a template, a CI step, a policy).
4. Verify with the escaped-defect metric that the level actually took.

The mistake is trying to jump from level 0 to level 4 in one motion. Maturity is habits, and habits are installed one at a time, backed by systems.

17.4 Regression signals

Teams slide backward silently. Watch for these:

- Review becomes a formality - “LGTM” without reading.
- Acceptance criteria drift back to adjectives (“make it robust”).
- Verification is skipped “this once, to ship on time.”
- The summary is cited as evidence in a decision meeting.
- Publication defaults creep back to permissive.

Any one of these is a signal to re-anchor at the level you thought you had left.

Key takeaways. Maturity is measured by defects caught before shipping. The five levels run from ad hoc to verified, each defined by a signature behavior. Advance one level at a time by installing a guardrail that makes the new behavior a default, and watch for the quiet regression signals that undo progress.

Part IV - Reference

Chapter 18. Templates

Learning objectives. After this chapter you should be able to: (1) use the specification template directly; (2) use the acceptance-criteria template; (3) run the review checklist; (4) record a blocker in the standard form.

18.1 Task specification template

Copy and fill in. Every field should be filled before delegation; leave none blank.

Objective

<One falsifiable sentence: what is true when the work is done.>

Scope

<Files/modules that may be changed.>

Do not change

<Invariants, public APIs, behavior to preserve, files out of bounds.>

Publication intent

<None (default) | commit | push | PR - and to which branch.>

Acceptance criteria

1. <Executable check: command + expected result.>
2. <Executable check.>
3. <Executable check.>

Output contract

<What to report back: change summary, evidence, exit codes, limitations.>

18.2 Acceptance-criteria template

Each criterion must be falsifiable, observable, and decidable (Section 7.4). Phrase each as:

“When I run <command>, I observe <result>.”

Example: “When I run `make test`, all tests pass and no test file was modified.” Example: “When I run `git diff -- src/auth/index.ts`, the output is empty.” If you cannot fill in <command> and <result>, the criterion is not done.

18.3 Review checklist

Reproduced from Section 8.6 for daily use:

- ☐ I read the full diff, not the summary.
- ☐ Every changed file is within the authorized mutation scope.
- ☐ I can state, from the source, what the change does and what it might break.
- ☐ Behavior changes, if any, are intentional and specified.
- ☐ Error paths and edge cases are handled, not just the happy path.
- ☐ No security-relevant change (dependency, permission, secret, validation) escaped notice.
- ☐ Tests were strengthened or correctly updated, never weakened to pass.
- ☐ My material inferences were confirmed against the source, not assumed.

18.4 Blocker record template

Four fields, always (Section 11.6):

Goal

<The objective that remains unmet.>

Route attempted

<The approach, decomposition, or technique just tried.>

Observed failure

<The exact error or the exact unmet criterion, quoted.>

Next route planned

<The materially different approach to try next, or "escalate to human" with reason.>

Key takeaways. The four templates - specification, acceptance criteria, review checklist, and blocker record - are the working surfaces of the discipline. Use them verbatim until the habits they encode are automatic.

Chapter 19. Worked Case Studies

Learning objectives. After this chapter you should be able to: (1) follow the lifecycle through four representative tasks; (2) identify where each task went right or wrong; (3) apply the lessons to your own work.

19.1 The isolated bug fix

Task. A payment webhook intermittently double-processes a retry.

Specification. Outcome: a retry is processed at most once (idempotency). Mutation intent: `src/webhooks/*` only; the public API must not change. Publication: none. Criteria: (1) a new test that replays the same event twice and asserts a single processing; (2) the full suite passes; (3) the idempotency key is stored in the same transaction as the processing.

What review caught. The agent added an idempotency table but queried it *outside* the transaction, leaving a race window. Review of the diff - not the summary - revealed the check-then-act ordering; the fix moved the lookup inside the transaction. The bug would have shipped on the agent's word.

Lesson. The acceptance criterion “at most once” was only as strong as the review that caught the race the spec did not fully spell out. Specification and review are complementary, not redundant.

19.2 The new feature

Task. Add rate limiting to an API.

Specification. Outcome: per-key limits of N requests per minute, with a 429 and a `Retry-After` header when exceeded. Mutation intent: `src/api/*`, `src/rate/*`. Publication: PR to `feature/rate-limit`. Criteria: (1) tests for under-limit, at-limit, and over-limit; (2) the 429 path includes the header and does not count against other keys; (3) load test shows no meaningful p99 regression.

What went right. The spec named the failure path (the 429 and header), so the agent implemented and tested it rather than inventing its own behavior. Verification reproduced the header and the per-key isolation independently.

Lesson. Naming the failure behavior in the spec is what kept the agent from silently choosing a different, plausible-but-wrong error contract.

19.3 The refactor

Task. Extract a shared utility from three duplicated implementations.

Specification. Outcome: one canonical helper, three call sites migrated. Mutation intent: `src/util/*` and the three call sites only. Constraint: external behavior must be byte-for-byte unchanged. Criteria: (1) the full suite passes unchanged; (2) the three call sites produce identical outputs on a differential test against the old code; (3) no public API changed.

What nearly went wrong. The agent “cleaned up” a fourth, unrelated module in passing. The diff review caught the out-of-scope edit and it was reverted; the task was kept to its scope.

Lesson. The preserve-behavior constraint plus the scope list made the scope creep visible the moment it happened. Collateral edits are caught by the mutation boundary, not by goodwill.

19.4 The correctness-critical algorithm

Task. A scheduler must never double-book a resource.

Specification. Outcome: prove the no-double-booking invariant holds for the new scheduler. Mutation intent: `src/sched/*`. Criteria: (1) the invariant is stated as a precise proposition; (2) a machine-checked proof of the invariant exists and compiles; (3) property tests fuzz the boundary as a sanity net.

What went right. Because the claim was “always,” the team did not rely on tests alone; they required a proof, and tests served as a sanity net, not the evidence. The proof caught an off-by-one in the interval-overlap check that tests had missed.

Lesson. Match evidence to claim. “Never” and “always” are proven, not sampled. This is the Level-4 behavior of the maturity model (Chapter 17).

19.5 The failure case study

Task. “Clean up the auth module” - the vague request from Section 7.6.

What went wrong, played straight. The agent reformatted files across the repository, deleted two “unused” functions that were in fact used by a downstream service, and pushed to main because publication was never forbidden. The diff was unreadable, the breakage surfaced in production, and nobody could reconstruct what was intended.

What would have prevented it. A specification with a scope, a preserve-behavior constraint, a publication default of none, and executable acceptance criteria - the very transformation shown in Section 7.6. Every one of the four failures maps to a missing component of the specification.

Lesson. The cost of a vague specification is paid in full at review, verification, and production - multiplied by the agent’s speed. Cheap to prevent, expensive to repair.

Key takeaways. Four successful cases show the lifecycle working: review catching a race the spec missed, a spec naming failure behavior, the mutation boundary catching scope creep, and proof matching a “never” claim. One failure case shows a vague spec compounding into a production incident - and that every failure traced to a missing specification component.

Chapter 20. The Working Checklist

Learning objectives. After this chapter you should be able to: (1) run the five-phase checklist on any task; (2) use it as a team standard; (3) recognize that a skipped box is a risk, not a formality.

20.1 Before delegating

- ☐ The outcome is one falsifiable sentence.
- ☐ Mutation intent names the scope and the invariants to preserve.
- ☐ Publication intent is stated (default: none).
- ☐ Acceptance criteria are executable checks.
- ☐ Each criterion is falsifiable, observable, and decidable.
- ☐ The specification could be checked by a third party without questions.

20.2 While the agent works

- ☐ The agent is working within the delegated scope.
- ☐ The agent reads source before editing.
- ☐ High-consequence actions are gated on human authorization.
- ☐ Failures are being classified, not just re-prompted.

20.3 Before accepting

- ☐ I read the full diff, not the summary.
- ☐ The review checklist (Section 18.3) is complete.
- ☐ Every acceptance criterion has a passing, inspected check.
- ☐ Security-relevant changes were reviewed.
- ☐ Tests were not weakened.

20.4 Before publishing

- ☐ Publication is explicitly authorized for this action.
- ☐ The diff was inspected after the final mutation.
- ☐ Secrets were scanned and the scan is clean.
- ☐ The change is on the intended branch, not a shared mainline by accident.

20.5 On failure

- ☐ The failure is classified (spec / verification / route / capability / goal).
- ☐ The next attempt differs materially from the last.
- ☐ The blocker is recorded (goal, route, failure, next route).
- ☐ After the bounded attempts, escalate rather than loop.

Key takeaways. The checklist is the discipline in five phases: before delegating, while the agent works, before accepting, before publishing, and on failure. A skipped box is a known, named risk - which is the point of having the box.

Chapter 21. Glossary

Agent. A system that plans and executes toward a goal, using tools, with varying autonomy. In this book, the AI system that writes, tests, and iterates on software.

Agentic engineering. The discipline of specifying software outcomes precisely, delegating implementation and iteration to an AI agent, and accepting results only when source and execution evidence satisfy explicit acceptance criteria.

Acceptance criteria. The executable, falsifiable checks that a change must pass to be accepted.

Capable but unaccountable. The defining asymmetry of the agent: it can do the work but cannot bear the consequence, so it is never the authority on its own correctness.

Completion era. The stage of AI-assisted coding where the AI autocompletes while the human still writes and decides.

Delegation. Handing a specification to the agent with explicit bounds and authorization, within which it may run autonomously.

Diff. The minimal complete record of what changed in a set of files; the primary artifact of review.

Escaped defect. A defect that ships to users; the core quality metric of the paradigm.

Mutation intent. The part of the specification stating what may change and what must not.

Publication intent. The part of the specification stating whether commit, push, or pull request is permitted.

Route failure. A failure of a particular approach, distinct from a failure of the goal itself; the default interpretation of failure.

Spec-driven development. A closely related term for agentic engineering emphasizing the written specification as the interface between human and agent.

Trust ladder. The graduated path from default skepticism to recorded acceptance: inspect, reproduce, cross-check, accept.

Vibe coding. Informal, conversational delegation with light review; the prototyping layer of the spectrum, not the production standard.

Verification. The production and inspection of executable evidence that a change meets its acceptance criteria.

Chapter 22. Further Reading

This book is self-contained, but the surrounding literature deepens each chapter. The list below is organized by topic; all are stable, foundational references rather than time-sensitive links.

- **On the paradigm shift.** Karpathy’s writing on “vibe coding” and the software 3.0 framing; literature on the division of labor between human judgment and automated generation.
- **On specification.** Classic requirements-engineering texts on writing testable, unambiguous requirements; the “specification by example” and behavior-driven development literature.
- **On review and defect detection.** Empirical software-engineering studies of code review effectiveness and defect injection; the literature on why reviewers miss certain bug classes.
- **On verification and proof.** The testing-versus-proofing literature; introductions to interactive theorem proving and formal verification (Lean 4, Coq, TLA+); property-based testing (QuickCheck, Hypothesis).
- **On security.** OWASP guidance on dependency, secret, and injection risks; prompt-injection literature from adversarial machine learning.
- **On teams and process.** The metrics literature distinguishing activity metrics from outcome metrics (e.g., the DORA four keys, escaped-defect rate).
- **On reasoning under uncertainty.** Calibration literature showing why expressed confidence is a poor proxy for correctness; the epistemology of expert judgment.

Treat this list as pointers to the intellectual foundations of each chapter, not as a reading assignment to complete before practicing. Practice the lifecycle; consult the literature when a chapter’s discipline needs deeper grounding.

Appendices

Appendix A - Example Specification Documents

This appendix provides three complete, ready-to-adapt specifications. They illustrate the four components - outcome, mutation intent, publication intent, acceptance criteria - at different scales.

A.1 A bug fix

Objective

A retried payment webhook is processed at most once, so no customer is double-charged.

Scope

src/webhooks/*

Do not change

The public webhook handler signature; any file outside src/webhooks; existing tests.

Publication intent

None. Stop at a local change for review.

Acceptance criteria

1. `pytest test/webhooks/test\_idempotency.py` passes; the new test replays the same event twice and asserts exactly one processing.
2. `make lint && make build` exit 0.
3. `make test` passes with no test file modified.
4. The idempotency key lookup and the processing write occur in the same transaction (verify by reading the diff: no separate read before the transaction).

A.2 A new feature

Objective

The public API enforces per-key rate limits of 100 requests/minute with a 429 and a Retry-After header when exceeded.

Scope

src/api/*, src/rate/*, test/api/*

Do not change

The wire format of successful responses; the public API of any other module.

Publication intent

Pull request to feature/rate-limit only; no merge.

Acceptance criteria

1. ``pytest test/api/test_rate_limit.py`` passes; it covers under-limit, at-limit, and over-limit, and asserts the 429 and Retry-After header.
2. The over-limit response for key A does not consume key B's allowance.
3. ``make test`` passes with no test file modified.
4. A load test at 2x the limit shows no p99 latency regression beyond 10%.

A.3 A correctness-critical change

Objective

The scheduler provably never double-books a resource.

Scope

src/sched/*

Do not change

The public scheduler interface; the persistence schema.

Publication intent

None. Stop at a local change.

Acceptance criteria

1. The no-double-booking invariant is stated as a precise proposition:
for any two accepted bookings overlapping in time, their resources are distinct.
2. A machine-checked proof of the invariant compiles (exit code 0) with no ``sorry``, ``admit``, or ``axiom``.
3. Property tests fuzz interval overlaps as a sanity net and pass.

Appendix B - Failure-Mode Catalog

A compact reference of the failure modes introduced throughout the book, with the chapter that treats each.

Failure mode	Symptom	Chapter
Confident wrong output	Correct-looking code that violates the spec	2, 5
Silent gap-filling	Agent invents an unspecified requirement	5, 7
Summary trust	Acceptance based on the agent's prose	8
Scope creep	Edits outside the mutation intent	8, 10
Covered behavior change	A "refactor" alters an edge case	8
Test weakening	Assertions deleted or loosened to pass	9

Failure mode	Symptom	Chapter
Secret leakage	Credential in code, log, or diff	13
Prompt injection	Instructions smuggled in untrusted input	13
Equivalent retry	Re-running the same failing approach	11, 16
Unbounded autonomy	Publication or mutation without authorization	10, 16
Delegated intent	Agent asked to decide what the product should be	3, 16

Each entry has a correction in the referenced chapter. When a failure surprises you, find its row here and re-read the chapter.

Appendix C - Exercises

Exercises are grouped by part. They are designed to be done against a real repository and a real agent.

C.1 Foundations

1. Write the three-eras table (Section 1.2) from memory, and explain which scarce skill moves to the human in the agentic era.
2. In one sentence, define agentic engineering without using the words “AI”, “prompt”, or “tool”.
3. State the capable-but-unaccountable asymmetry and give a concrete example of an agent claim that must not be trusted on its face.
4. Place your current workflow on the spectrum of Section 2.2, and justify the placement with evidence from your last three tasks.

C.2 The lifecycle

5. Take a task you completed last week and rewrite its (implicit) specification using the Section 18.1 template. Identify what was previously missing.
6. For a real change, run the trust ladder (Section 5.2) explicitly and record what each rung required.
7. Write acceptance criteria for “make the login page faster” that pass the verifiability test of Section 7.4.
8. Review a real agent-produced diff and complete the Section 18.3 checklist; note every box you could not honestly check.

C.3 Practice

9. Classify five recent agent failures using the table in Section 11.2, and state the materially different retry for each.
10. Draft a prompt with objective, scope, constraints, authorization, criteria, and output contract for a task you do repeatedly; then run it and record where the agent stumbled.

11. Scan a recent diff for the six security-relevant changes of Section 13.2 and report what you found.
12. Audit your team against the maturity model (Chapter 17) and name the single next-level behavior to install.

C.4 Reference

13. Using the Section 18.1 template, write specifications for a bug fix, a feature, and a refactor, and exchange them with a peer to check third-party verifiability.
14. From the failure catalog (Appendix B), pick the three failure modes you have personally experienced and write a one-paragraph postmortem for each using the blocker record (Section 18.4).
15. Run the full working checklist (Chapter 20) on one end-to-end task and write a short retrospective on which boxes were hardest to satisfy.

Key takeaways. Exercises turn the reading into practice. Do them against real work, not invented examples - the discipline is learned in the friction of a real diff, a real failure, and a real acceptance decision.

This book is a living document. As models, tools, and practices change, the principles in Part I - own intent, specify first, review as evidence, verify before accepting, control mutation and publication, and treat failure as route failure - are intended to remain stable. Revise the practices; keep the principles.

References

The foundational paradigm content in this book — the paradigm definition, the role shift, and the verification discipline — is grounded in the practitioner and preprint sources listed below. Sources [1]–[6] are cited throughout the body with bracketed markers and are reproduced here with URLs for traceability. Sources [7] and [8] document the source-grounded textbook-assembly pipeline used to produce this book itself, and are cited here as methodology rather than as in-text claims.

1. **Vibe Coding & AI Development 2026: Cursor, Copilot, Claude Code** — BraivIQ blog. <https://www.braiviq.com/blog/vibe-coding-ai-development-2026-cursor-copilot-claude-code>
2. **Agentic AI in Software Development: From Vibe Coding to Spec-Driven Development** — Infosys blog. <https://blogs.infosys.com/emerging-technology-solutions/artificial-intelligence/agentic-ai-in-software-development-from-vibe-coding-to-spec-driven-development.html>
3. **Agentic Engineering Explained** — Taskade blog. <https://www.taskade.com/blog/agentic-engineering-explained>
4. **Spec-Driven Development** — AI Wiki. https://aiwiki.ai/wiki/spec_driven_development
5. **Grounding and Hallucination Control** — Nemorize roadmap. <https://nemorize.com/roadmaps/2026-modern-ai-search-rag-roadmap/lessons/grounding-hallucination-control>
6. **Vibe Coding vs. Agentic Coding** — arXiv preprint. <https://www.arxiv.org/pdf/2505.19443>
7. **The pipeline for assembling source-grounded textbooks** — arXiv preprint. <https://arxiv.org/html/2607.28109v1>
8. **Augmenting textbooks with generative AI** — arXiv preprint. <https://arxiv.org/html/2509.13348v3>